Reviewer Pack — Minimal Representation Rank and Coxeter System Index of Bool…

Entry 0aa52fbea438 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 08:38:03 UTC

Minimal Representation Rank and Coxeter System Index of Boolean Functions

Entry ID: `0aa52fbea438` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-22 08:38:03 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory: Coxeter Systems **Field B** (complexity object): Complexity Theory: Boolean Function Complexity

Statement:

> {'sentence_1': 'For any Boolean function f with n variables, the minimal representation rank ρ(f) over a Coxeter system is at least as large as its communication complexity C(f).', 'sentence_2': 'That is, for all n ≥ 3, ρ(f) = Θ(C(f)).', 'sentence_3': 'Furthermore, there exists a constructive mapping from f to a Coxeter system representation that preserves the communication complexity.'}

Rationale (proposer's reasoning):

> {'sentence_1': 'This conjecture proposes a connection between the algebraic structure of Coxeter systems and the communication complexity of Boolean functions, which could reveal new insights into the computational hardness of boolean functions.', 'sentence_2': 'Representation theory has been used to study algebraic properties of complex systems, and exploring its application in complexity theory might expose inherent structures that are not visible with current methods.', 'sentence_3': 'Coxeter groups often appear in combinatorial problems related to symmetry and structure. Applying this mathematical tool to boolean functions may lead to new ways of understanding their computational complexity.'}

Taxonomy category: `GCT_DET_PERM` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `c7b9d1b8bd3e0d0e`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For each Boolean function f, if the ratio of minimal representation rank ρ(f) over Coxeter system index C(f) is ≤ 1.5 across all seeds and their mean is ≥ 0.8, the conjecture is supported.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.95	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_boolean_function(n):
    return [random.randint(0, 1) for _ in range(2**n)]

def communication_complexity(f):
    n = int(math.log2(len(f)))
    max_comm_cost = 0
    for i in range(2**n):
        for j in range(i+1, 2**n):
            if f[i] != f[j]:
                comm_cost = bin(i ^ j).count('1')
                if comm_cost > max_comm_cost:
                    max_comm_cost = comm_cost
    return max_comm_cost

def minimal_representation_rank(f):
    n = int(math.log2(len(f)))
    rank = 0
    for i in range(2**n):
        if f[i] == 1:
            rank += 1
    return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    total_comm_cost = 0
    total_rank = 0
    instances_tested = 0
```

```
    for n in n_values:
        f = generate_boolean_function(n)
        comm_cost = communication_complexity(f)
        rank = minimal_representation_rank(f)
        total_comm_cost += comm_cost
        total_rank += rank
        instances_tested += 1

    mean_comm_cost = Fraction(total_comm_cost, instances_tested)
    mean_rank = Fraction(total_rank, instances_tested)
    ratio = mean_rank / mean_comm_cost

    conjecture_holds = ratio <= Fraction(3, 2) and ratio >= Fraction(1, 2)
    counterexample = "" if conjecture_holds else f"Ratio {ratio} outside [0.5, 1.5]"

    return {
        "metric_name": "Ratio of Minimal Representation Rank to Communication Complexity",
        "metric_value": float(ratio),
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    if len(sys.argv) > 1:
        seeds = [int(s) for s in sys.argv[1:]]
    else:
        primes = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73,
        seeds = random.sample(primes, min(30, len(primes)))

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_ratio = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_ratio} std=0 support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"Ratio outside [0.5, 1.5]\" first_failing_seed={
    else:
        print("RESULT: INCONCLUSIVE insufficient support")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test timed out before producing data, which means we cannot verify the conjecture's support or falsification. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14971
2	preregistration	ollama_remote	glm4:latest	0	0	9196
3	novelty	ollama_remote	glm4:latest	0	0	9965
4	novelty	ollama_remote	glm4:latest	0	0	8568
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11808
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11560
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8617
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10314
9	judge	ollama_remote	glm4:latest	0	0	21343

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 106343 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/0aa52fbea438.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0aa52fbea438.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/0aa52fbea438.tar.gz` (if generated)